

Week5 Session 1- Facilitation Guide

(Introduction to Node.js)

Index

- I. **Recap of the previous session**
- II. **What is Node.js?**
- III. **Features & Benefits of Node.js**
- IV. **Node.js vs. JavaScript (Browser vs. Server)**
- V. **Installing Node.js and NPM**
- VI. **Simple Node.js Program**

(2hr ILT)

I. Recap of the previous session

In the last sessions, we discussed the core concepts of web development, focusing on front-end technologies. These topics laid the groundwork for building visually appealing and interactive web applications.

As a quick recap, let's see the key points:

HTML

HTML is the backbone of web pages and is used to structure content.

- It includes elements and tags like <div>, <p>, <h1>-<h6> (headings), and <a> (links) to define the layout and content of a webpage.
- We also explored text formatting tags like (bold), <i> (italics), and <u> (underline) to enhance the presentation of content.

CSS

CSS is used to style and layout web pages, ensuring they are visually appealing.

- It controls colors, fonts, spacing, positioning, and responsiveness.
- We discussed the use of inline, internal, and external stylesheets for applying styles.
- Key features included selectors, box models, and pseudo-classes to create sophisticated designs.

Bootstrap

Bootstrap is a CSS framework that simplifies responsive design and styling.

- It provides pre-designed components like buttons, modals, navigation bars, and cards.
- Its grid system ensures mobile responsiveness, enabling faster and more consistent development.

JavaScript

JavaScript brings interactivity to web pages.

- We covered variables, functions, loops, and conditionals, which form the foundation of JavaScript programming.
- Explored DOM manipulation, event handling, and dynamic updates to create interactive elements like forms, animations, and basic API calls.

Transition to Node.js

Having a solid understanding of JavaScript, we're now ready to explore Node.js, a powerful runtime environment that enables server-side programming with JavaScript. Node.js allows us to build scalable and efficient backend applications, making it an essential skill for full-stack development.

In this session we will learn about node.js installation with its features.

II. What is Node.js?

Node.js is a JavaScript runtime environment that allows developers to execute JavaScript code on the server side. Built on the V8 JavaScript engine (the same engine used in Google Chrome), Node.js provides an efficient and scalable platform for building server-side and networking applications.

It was introduced by Ryan Dahl in 2009 and revolutionized web development by enabling developers to use JavaScript not only for front-end development but also for back-end development.

III. Features of Node.js

1. Asynchronous and Event-Driven

- Node.js uses an event-driven, non-blocking I/O model, making it lightweight and efficient.
- Tasks are executed asynchronously, meaning the server doesn't wait for one task to complete before moving to the next.

2. Fast and Scalable

- Powered by the V8 engine, Node.js executes JavaScript code extremely fast.
- Its single-threaded, event-driven architecture enables it to handle a large number of simultaneous connections with high throughput.

3. Single-Threaded with Event Loop

- Node.js uses a single thread to handle multiple requests via its event loop, which eliminates the need for creating multiple threads, thus reducing resource consumption.

4. Cross-Platform

- Node.js is compatible with multiple platforms like Windows, macOS, and Linux, making it versatile for developers.

5. NPM (Node Package Manager)

- Node.js comes with NPM, a vast ecosystem of open-source libraries and modules that developers can use to build applications faster.

6. Real-Time Applications

- Node.js is ideal for building real-time applications like chat applications, online gaming, and collaborative tools.

7. Built-in Libraries

- Node.js has built-in libraries to perform various tasks, such as creating an HTTP server without needing external dependencies.

Benefits of Node.js

1. Improved Performance

- The non-blocking I/O and asynchronous programming model allow it to handle multiple requests simultaneously, reducing latency and increasing performance.

2. Efficient Development

- Developers can use JavaScript on both the client and server side, enabling full-stack development with a single language.

3. Scalability

- Node.js is designed to scale horizontally (by adding more instances) and vertically (by increasing the server's resources).

4. Rich Ecosystem

- With over a million libraries in NPM, developers can find modules for almost any task, saving time and effort in building applications from scratch.

5. Community Support

- A large and active community continuously contributes to Node.js, providing updates, tools, and libraries.

6. Real-Time Data Handling

- Node.js is well-suited for applications requiring real-time interactions, such as live chat, notifications, or collaborative editing tools.

7. Cost-Effective

- Its efficiency reduces server costs, and using a single language (JavaScript) across the stack reduces the need for additional developers specializing in different technologies.

8. Support for Microservices

- Node.js makes it easier to build modular, independent components that work together, ideal for microservices architecture.

When to Use Node.js?

Node.js is particularly suitable for:

- Real-Time Applications: Live chats, online gaming, or collaborative tools.
- APIs and Microservices: Building RESTful APIs or modular microservices.
- Streaming Applications: Audio/video streaming platforms.
- IoT Applications: Handling multiple device connections simultaneously.
- Single Page Applications (SPAs): Building dynamic, interactive web applications like Gmail.

Node.js is a game-changer for modern web development, combining speed, scalability, and efficiency, making it one of the most popular technologies for building backend applications today.

IV. Node.js vs JavaScript: A Detailed Comparison

Both Node.js and JavaScript are closely related, as Node.js is built on JavaScript. However, they serve different purposes and are used in distinct environments. Below is a detailed comparison of Node.js and JavaScript:

1. Definition

Aspect	Node.js	JavaScript
Definition	Node.js is a runtime environment that allows executing JavaScript code outside a web browser.	JavaScript is a scripting language primarily used for creating interactive elements in web browsers.
Purpose	Designed for server-side and back-end development.	Designed for client-side and front-end development.

2. Environment

Aspect	Node.js	JavaScript
--------	---------	------------

Execution	Executes JavaScript code in a server-side environment (e.g., a machine, server).	Executes JavaScript code in a client-side environment (e.g., web browsers like Chrome, Firefox).
Engine	Built on V8 JavaScript Engine, the same engine that powers Google Chrome.	Runs in browser environments (e.g., Chrome, Firefox, Safari) using their respective JavaScript engines.

3. Usage

Aspect	Node.js	JavaScript
Primary Use	Used for building back-end applications, RESTful APIs, microservices, and server-side scripting.	Used for building dynamic, interactive front-end web applications.
Applications	Applications such as real-time chat, video streaming, IoT, and API servers.	Adding interactivity to web pages (e.g., form validation, dynamic updates, animations).

4. Features and Libraries

Aspect	Node.js	JavaScript
Modules	Comes with built-in modules like fs (file system), http (for servers), and path.	Uses third-party libraries (e.g., jQuery, React) for additional features, but no built-in modules.
Global Objects	Provides server-side global objects like require, process, and module.	Provides browser-side global objects like window, document, and navigator.

5. Architecture

Aspect	Node.js	JavaScript
Programming Model	Asynchronous, event-driven, non-blocking I/O for high scalability.	Event-driven for client-side interactions, typically runs synchronously in browsers.
Multithreading	Single-threaded but uses libuv to handle asynchronous tasks.	Single-threaded, mainly designed for front-end tasks.

6. Key Features

Aspect	Node.js	JavaScript
File Handling	Can access and manipulate the file system using the fs module.	Cannot directly access the file system; requires back-end interaction.
Server-Side Scripting	Enables server-side scripting (e.g., creating servers, handling requests/responses).	Cannot handle server-side tasks directly.
I/O Capabilities	Can handle file I/O, stream data, and manage databases.	Limited to interacting with the DOM and making API calls.

7. Syntax

The syntax of JavaScript is the same in both Node.js and browsers, but the APIs available differ.

Aspect	Node.js Example	JavaScript Example
Hello World	<pre>javascript const http = require('http'); http.createServer((req, res) => { res.end('Hello World'); }).listen(3000);</pre>	<pre>javascript console.log('Hello World');</pre>
File Handling	<pre>javascript const fs = require('fs'); fs.writeFileSync('file.txt', 'Hello Node.js');</pre>	Not possible directly in JavaScript.
DOM Interaction	Not applicable in Node.js.	<pre>javascript document.getElementById('btn').addEv entListener('click', () => console.log('Clicked!'));</pre>

8. Tools and Ecosystem

Aspect	Node.js	JavaScript
Tools	Node Package Manager (NPM), Express.js, Koa.js.	Libraries like React.js, Vue.js, Angular for advanced front-end development.
Dependencies	Huge ecosystem of over a million open-source packages available in NPM.	Dependencies managed manually or through tools like Yarn or NPM when combined with Node.js.

9. Performance

Aspect	Node.js	JavaScript
Performance	Optimized for handling multiple requests using non-blocking I/O.	Limited to front-end tasks, performance depends on browser capabilities.
Concurrency	Ideal for handling real-time, concurrent user requests (e.g., chat apps, games).	Not optimized for high-concurrency tasks.

10. Real-Life Use Cases

Aspect	Node.js	JavaScript
Examples	- Backend of Netflix, LinkedIn, Uber, and PayPal.	- Frontend interactivity in Gmail, Facebook, and Twitter.

Node.js	JavaScript
---------	------------

A server-side runtime to execute JavaScript code on servers.	A client-side scripting language for dynamic and interactive web pages.
Focused on handling backend operations efficiently and scaling applications.	Focused on enhancing the user experience on the front-end.

While Node.js enables JavaScript to extend beyond the browser, both together form a powerful duo for full-stack development.

V. Installing Node.js and npm

Node.js comes bundled with npm (Node Package Manager), so installing Node.js will also install npm. To install Node.js and npm on windows are:-

Node.js has various versions. The recommended version is the latest stable version, as it contains major and significant changes. These changes include bug fixes and security updates, compatibility with your project dependencies, and so on.

Step 1: Install Node.js

To install Node, navigate to the Node.js website. On their webpage, you have the option to download either the recommended version or the current version, as seen in the image below:-



Step 2: Run the Installer

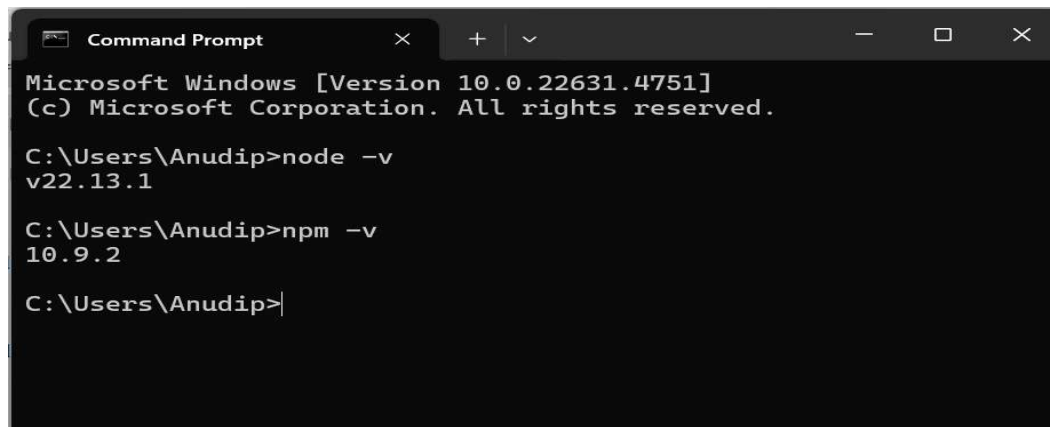
- Locate the downloaded .msi file and double-click to run it.
- Follow the prompts in the setup wizard, accept the license agreement, and use the default settings for installation.

Step 3: Verify Node.js Installation

Open Command Prompt or PowerShell > Check the installed versions by running these commands:

- Type `node -v` and press Enter to check the Node.js version.
- Type `npm -v` and press Enter to check the npm version.

Both commands should return version numbers, confirming successful installation.



```
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Anudip>node -v
v22.13.1

C:\Users\Anudip>npm -v
10.9.2

C:\Users\Anudip>
```

Step 4: Update npm(optional)

To ensure you have the latest npm version, run :

`npm install -g npm@latest`

VI. Simple Node.js program

We can create either console based or web based applications in Node.js.

Node.js console based app

Step 1 : Create any node.js file in any text editor and save it with name firstapp.js

Code Example :

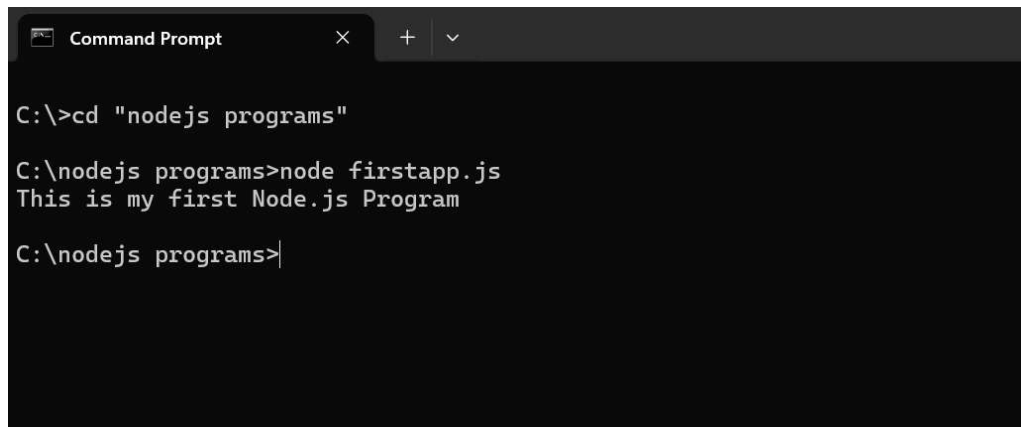
```
console.log("This is my first Node.js Program");
```

Step 2 : Open Command prompt and navigate to file location.

Step 3: Run the program

```
node firstapp.js
```

Output :



```
C:\>cd "nodejs programs"

C:\nodejs programs>node firstapp.js
This is my first Node.js Program

C:\nodejs programs>
```

Node.js web based app

A node.js web application contains the following three parts:

1. Import required modules: The first step is to use the require directive to load the http module and store returned HTTP instance into http variable. For example:

```
var http = require("http");
```
2. Create server: You have to establish a server which will listen to client's request similar to Apache HTTP Server.
3. Read request and return response: Server created in the second step will read HTTP requests made by client which can be a browser or console and return the response.

Step 1 : Create a node.js app with name secondapp.js

Code Example :

```
var http = require("http");
http.createServer(function (request, response) {
```

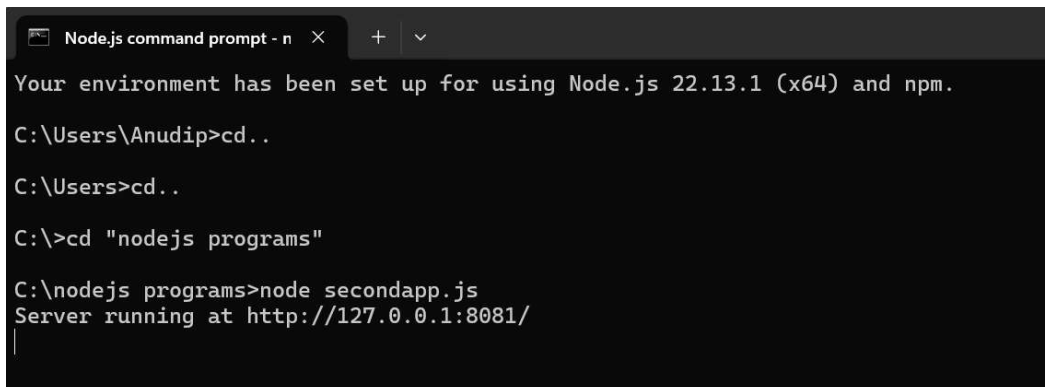
```
// Send the HTTP header
// HTTP Status: 200 : OK
// Content Type: text/plain
response.writeHead(200, {'Content-Type': 'text/plain'});
// Send the response body as "Hello World"
response.end('Hello World\n');
}).listen(8081);
// Console will print the message
console.log('Server running at http://127.0.0.1:8081/');
```

1. Import required module: The first step is to use `require()` directive to load the `http` module and store returned HTTP instances into `http` variable.
2. Create server: In the second step, you have to use created `http` instance and call `http.createServer()` method to create server instance and then bind it at port 8081 using the `listen` method associated with server instance. Pass it a function with `request` and `response` parameters and write the sample implementation to return "Hello World". For example:
3. Combine step1 and step2 together in a file named "secondapp.js".

Step 2 : Open Command prompt and navigate to file location.

Step 3: Run the program

node secondapp.js



```
Node.js command prompt - n  X  +  v
Your environment has been set up for using Node.js 22.13.1 (x64) and npm.
C:\Users\Anudip>cd..
C:\Users>cd..
C:\>cd "nodejs programs"
C:\nodejs programs>node secondapp.js
Server running at http://127.0.0.1:8081/
```

Now the server has started.

Make a request to Node.js server:

Open `http://127.0.0.1:8081/` in any browser. You will see the following result.

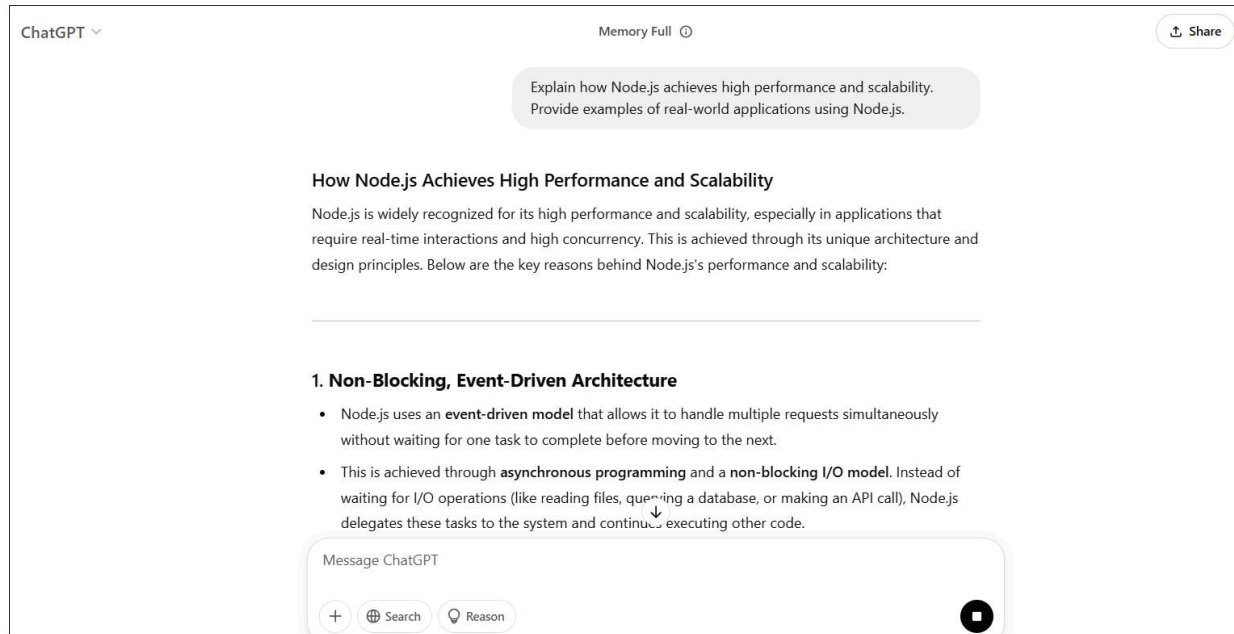


Exercise:

Use ChatGPT to explore how Node.js achieves high performance and scalability:

Put the below problem statement in the message box and see what ChatGPT says.

Explain how Node.js achieves high performance and scalability. Provide examples of real-world applications using Node.js.



After the 1.5 hr ILT, the student has to do a 30 minute live lab. Please refer to the Session 1 Lab doc in the LMS.